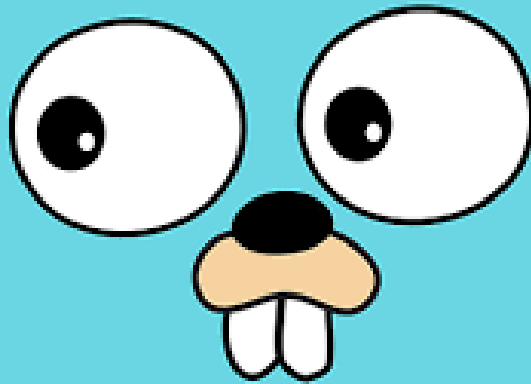




Advanced Concurrency Patterns



Discussion Points

- Worker pools
- Context package for cancellation
- Throttling and rate limiting





Worker pools

What is Worker pools?

The **Worker Pool Pattern**, also known as the **Goroutine Pool Pattern** in the context of Golang, is a concurrency design pattern used to manage and control a fixed number of worker routines (or threads) that handle a potentially unbounded number of tasks or jobs.

This pattern is helpful for limiting the number of concurrently running tasks, controlling resource usage, and improving application performance.

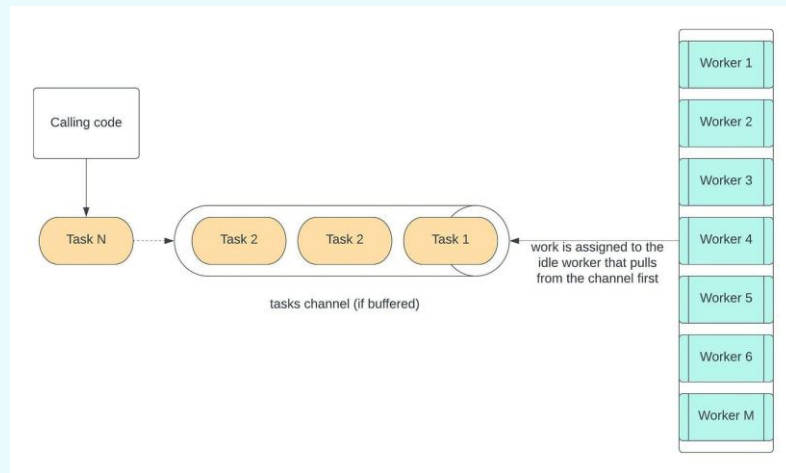




Worker pools

Core Concepts of the Worker pools?

- **Task Queue:** A channel or data structure where tasks (work units) are placed. This queue stores jobs that need to be executed by worker routines.
- **Workers:** These are goroutines or threads that fetch tasks from the task queue and execute them. Workers usually wait for new tasks when the queue is empty.
- **Job Dispatcher:** This component is responsible for distributing tasks to the worker pool. It manages the task queue and sends work to available workers.
- **Pool Size:** The number of workers that can run concurrently is limited to control resource usage (CPU, memory).

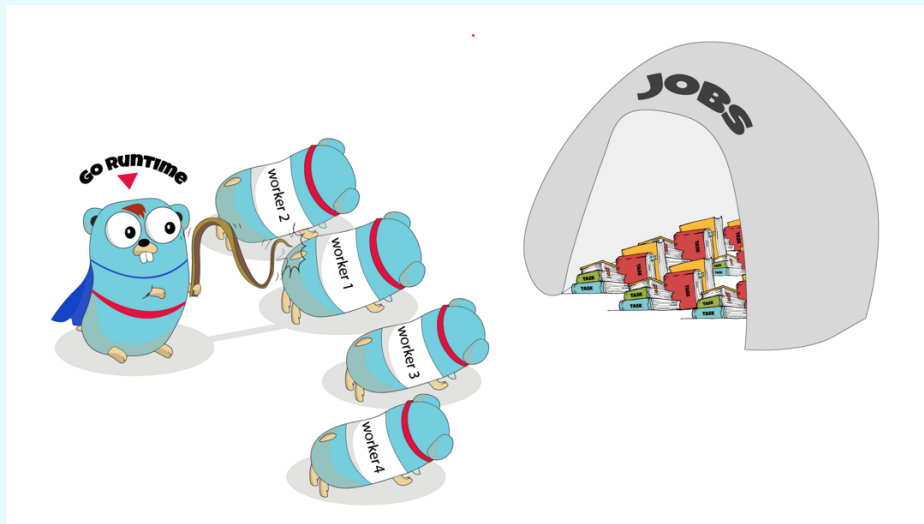




Worker pools

Demo Worker pools?

Let's have a look to bunch of Notifications than need to handle with limited resource...





Worker pools

Pros and Cons of Worker pools?

Advantages of the Worker Pool Pattern

- **Resource Control:** Limits the number of concurrent goroutines/threads, preventing system overload.
- **Throughput Optimization:** Efficiently handles a large number of tasks without overwhelming resources.
- **Load Balancing:** Tasks are evenly distributed among workers, balancing the workload.

Disadvantages of the Worker Pool Pattern

- **Idle Workers:** If the task rate is low, some workers might remain idle, wasting resources.
- **Complexity:** Introducing a worker pool can add complexity to the code structure.
- **Backlog:** If tasks are arriving faster than workers can process them, it can lead to a growing backlog of unprocessed tasks.





Context package for cancellation

What is Context package?

In Golang, the context package provides a way to manage request-scoped values, **cancellation** signals, and **timeouts/deadlines** across different parts of a program, particularly in concurrent or network-based applications. It's commonly used in server applications to control and synchronize the lifecycle of requests or tasks, allowing the code to handle cancellations or timeouts gracefully.





Context package for cancellation

Key features of Context package?

Cancellation Propagation: You can cancel an entire tree of tasks if any one task fails or is canceled. This is particularly useful for network requests, where you may want to stop processing if the client disconnects.

Timeouts and Deadlines: With context, you can set time limits on how long tasks are allowed to run, automatically stopping them if they exceed the allotted time.

Passing Values: context can carry request-scoped values, like authentication information, user IDs, or other metadata, across API boundaries.





Context package for cancellation

Types of Context package?

context.Background(): A base context that is typically used at the top level of your application or main function. It has no deadline, no values, and cannot be canceled.

context.TODO(): Often used when you're not yet sure which context to use or during the early stages of development.

WithCancel: Returns a new context that can be canceled manually. When you call `cancel()`, all tasks derived from this context will receive a cancellation signal.

WithDeadline: Sets a specific point in time when the context will be automatically canceled, useful for operations with hard time limits.

WithTimeout: Sets a relative duration after which the context will be canceled, commonly used for time-sensitive tasks.



Context package for cancellation

Demo of Context package?





Throttling and rate limiting

What is Throttling?

Throttling is the practice of controlling the speed at which requests or tasks are processed. It is often applied in situations where we need to limit the processing rate, for instance, to avoid resource exhaustion or system overload.

Example: An application may use throttling to process no more than 100 database transactions per second, even if more requests come in.

How It Works: Throttling is generally implemented using algorithms like *token bucket* or *leaky bucket*, which allow requests to be processed at a controlled rate, occasionally allowing bursts but smoothing out the flow over time.

Use Cases: Background processing, batch jobs, and tasks that involve high resource consumption often use throttling to keep resource usage in check.





Throttling and rate limiting

What is Rate Limiting?

Rate limiting is a technique to cap the number of requests or actions a client or user can make within a specified time frame. It is commonly used to prevent abuse, ensure fair usage, and protect APIs and services from high traffic spikes.

Example: An API might limit users to 100 requests per minute. If a user exceeds this limit, they will receive an error response, like HTTP 429 (Too Many Requests).

How It Works: Rate limiting is often implemented using sliding window or *fixed window algorithms*. Each client's request count is tracked over a defined time window, and if the limit is reached, further requests are rejected or delayed.

Use Cases: API gateways, public web services, and distributed systems commonly use rate limiting to prevent abuse, reduce denial-of-service (DoS) attack risks, and ensure consistent performance.



Thank you. Questions?

“Learn from yesterday, live for today, hope for tomorrow. The important thing is not to stop questioning.”

- **Albert Einstein**

